Міністерство освіти і науки України Державний університет «Житомирська політехніка» Факультет: ІКТ

Кафедра: Комп'ютерної інженерії Група КІ-26-1

Програмування

Звіт з лабораторної роботи №2

Тема: Розгалужені та циклічні обчислювальні процеси у мові С, консольні меню та оформлення інженерної звітності

Виконав: студент 1 курсу, групи КІ-26-1, Ничипорчук Ілля Віталійович

Перевірив: старший викладач кафедри комп'ютерної інженерії та штучного інтелекту Лисенко Максим Сергійович

Варіант 5

Житомир 2026

Посилання на GitHub:

https://github.com/IlliaNichiporchuk/programing-sem1

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.F7.5.000 –Лр.2		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Ничипорчук І.В			Звіт з лабораторної роботи №2	Літ.	Арк. 1 / Акрушів
Перевір.		Лисенко М.С					
Реценз.						ФІКТ, гр. КІ-26-1	
Н. Контр.							
Зав.каф.		Лобанчикова Н.М					

2. Мета роботи та індивідуальне завдання

Мета роботи

Набути практичних навичок проектування комбінованих алгоритмів із використанням операторів розгалуження (if-else) та циклів (for, while). Опанувати чисельні методи наближеного обчислення функцій за допомогою степеневих рядів та рекурентних співвідношень. Закріпити навички роботи з системою контролю версій Git та оформлення технічної документації у форматі PDF.

Завдання 1. Табулювання кусочно-неперервної функції.

Розробити програму мовою C, яка запитує межі відрізка [Xstart, Xend] та крок табулювання dX, перевіряє коректність введених даних (Xstart < Xend, dX > 0) і виводить таблицю значень функції Y(X) на відрізку, де функція задана кусочно:

$X \le -2$: $Y = 4.0$;

$-2 < X < 2$: $Y = X^2 - 1$;

$X \ge 2$: $Y = 2X$.

3. Математичний аналіз та рекурентні співвідношення

3.1. Завдання 1. ОДЗ і точки розриву

Усі три гілки функції (константа, $X^2 - 1$, $2X$) є многочленами, тому визначені для всіх дійсних X. ОДЗ: $X \in (-\infty; +\infty)$. Ділення на нуль і корінь з від'ємного числа не виникають, тому повідомлення «[Не визначено]» не з'являється.

Розриви можливі лише в точках стику гілок: X = −2: ліва границя 4.0, права $(-2)^2 - 1 = 3$, розрив першого роду (стрибок −1);
X = 2: ліва границя $2^2 - 1 = 3$, права $2 \cdot 2 = 4$, розрив першого роду (стрибок 1).

3.2. Завдання 2. Область збіжності та рекурентне співвідношення

$1/(1 - X) = \Sigma X^n$, n = 0…∞. Для члена $a_n = X^n$ маємо $\lim |a_n / a_{n-1}| = |X|$, тому ряд збігається при $|X| < 1$.
Рекурентне співвідношення:
$R(n, X) = a_n / a_{n-1} = X^n / X^{n-1} = X$,
тобто $a_n = a_{n-1} \cdot X$, $a_0 = 1$. Факторіалів у ряді немає, степінь скорочується повністю. У програмі це рядок term *= x;.

4. Опис алгоритмів і логічних рішень

4.1. Завдання 1

Програма зчитує Xstart, Xend і dX функцією scanf, перевіряючи її повернене значення: якщо введено не число, виводиться повідомлення про помилку і програма завершується з кодом 1. Далі перевіряється умова Xstart < Xend та dX > 0. Якщо вона не виконується, програма також виводить помилку і завершується з ненульовим кодом.

Кількість кроків табулювання обчислюється наперед за формулою floor((Xend − Xstart) / dX + 1e-9), а поточне значення визначається як X = Xstart + i · dX. Так уникається накопичення похибки дійсних чисел при багаторазовому додаванні кроку.

На кожній ітерації циклу for конструкція if-else визначає інтервал: якщо $X \leq -2$, то $Y = 4.0$; інакше, якщо $X < 2$, то $Y = X^2 - 1$; в іншому випадку $Y = 2X$. Результат виводиться в таблицю з фіксованою шириною колонок (%10.2f та %12.4f).

4.2. Завдання 2

Програма зчитує X та точність ε. Захист вхідних даних: перевіряється повернене значення scanf, умова належності до області збіжності $|X| < 1$ та умова $\varepsilon > 0$. У разі порушення будь-якої з них виводиться повідомлення про помилку і програма завершується.

Сума накопичується в циклі while, доки $|a_n| \geq \varepsilon$. Кожен наступний доданок обчислюється через рекурентне співвідношення term *= x, без виклику pow() та обчислення степеня на кожній ітерації.

Захист від зациклення (Timeout Guard): у циклі підраховується кількість ітерацій. Якщо вона досягає 100 000, а потрібна точність не досягнута, цикл примусово переривається командою break і виводиться попередження. Після циклу виводяться сума ряду, точне значення $1/(1 - X)$, абсолютна та відносна похибки і кількість ітерацій.

5. Лістинг вихідного коду

5.1. Лістинг програми task1.c

```c
#include <stdio.h>
#include <math.h>
int main(void) {
   const double EPS = 1e-9;
     const double LEFT_BOUND = -2.0;
     const double RIGHT_BOUND = 2.0;
     const double CONST_VALUE = 4.0;

     double x_start, x_end, step;

     printf("Enter Xstart: ");
     if (scanf("%lf", &x_start) != 1) {
         printf("Error: Xstart must be a number.\n");
         return 1;
     }
     printf("Enter Xend: ");
     if (scanf("%lf", &x_end) != 1) {
         printf("Error: Xend must be a number.\n");
         return 1;
     }
     printf("Enter step dX: ");
     if (scanf("%lf", &step) != 1) {
         printf("Error: dX must be a number.\n");


         return 1;
    }

      if (x_start >= x_end || step <= 0) {
        printf("Error: require Xstart < Xend and dX > 0.\n");
          return 1;
     }

       printf("\n%10s | %12s\n", "X", "Y(X)");
       printf("-----------+------------\n");

     long steps = (long)floor((x_end - x_start) / step + EPS);
     for (long i = 0; i <= steps; i++) {
     double x = x_start + i * step;
     double y;
```

```
    if (x <= LEFT_BOUND) {
        y = CONST_VALUE;
    } else if (x < RIGHT_BOUND){
        y = x * x - 1.0;

    } else {
         y = 2.0 * x;
    }

      printf("%10.2f | %12.4f\n", x, y);
    }

    return 0;
}
```

5.2. Лістинг програми task2.c

```
#include <stdio.h>
#include <math.h>

int main(void) {
    const int MAX_ITERATIONS = 100000;

    double x, eps;

    printf("Enter X (|X| < 1): ");
          if (scanf("%lf", &x) != 1) {
              printf("Error: X must be a number.\n");
              return 1;
          }
           if (fabs(x) >= 1.0) {
                printf("Error: X is outside the convergence region |X| < 1.\n");
                return 1;
          }

          printf("Enter precision eps (e.g. 1e-5): ");
          if (scanf("%lf", &eps) != 1 || eps <= 0) {
              printf("Error: eps must be a positive number.\n");
              return 1;
           }
               double term = 1.0;
               ouble sum = 0.0;
```

```
    int iterations = 0;

    while (fabs(term) >= eps) {
        if (iterations >= MAX_ITERATIONS) {
            printf("Warning: precision not reached in %d iterations.\n",
                   MAX_ITERATIONS);
            break;
        }
        sum += term;
        term *= x;
        iterations++;
    }

    double exact = 1.0 / (1.0 - x);
    double abs_error = fabs(exact - sum);
    double rel_error = abs_error / fabs(exact);

    printf("Series sum             : %.10f\n", sum);
    printf("Exact 1/(1-X)          : %.10f\n", exact);
    printf("Absolute error       : %.3e\n", abs_error);
    printf("Relative error         : %.3e\n", rel_error);
    printf("Iterations               : %d\n", iterations);

    return 0;
}
```

6. Результати тестування та верифікація

6.1. Завдання 1. Таблиця табулювання

Вхідні дані: Xstart = –5, Xend = 5, dX = 1.

```
Enter Xstart: -5
Enter Xend: 5
Enter step dX: 1

         X |         Y(X)
-----------+-------------
     -5.00 |       4.0000
     -4.00 |       4.0000
     -3.00 |       4.0000
     -2.00 |       4.0000
     -1.00 |       0.0000
      0.00 |      -1.0000
      1.00 |       0.0000
      2.00 |       4.0000
      3.00 |       6.0000
      4.00 |       8.0000
      5.00 |      10.0000
```

6.2. Завдання 1. Некоректні вхідні дані

Вхідні дані: Xstart = 0, Xend = 5, dX = –1. Програма виводить повідомлення про помилку і завершується з ненульовим кодом.

```
PS C:\Users\illia\projects\programming-sem1\lab02> .\task1.exe
Enter Xstart: 0
Enter Xend: 5
Enter step dX: -1
Error: require Xstart < Xend and dX > 0.
```

6.3. Завдання 2. Збіжність ряду

Точне значення функції для X = 0.5: 1/(1 – 0.5) = 2.

```
PS C:\Users\illia\projects\programming-sem1\lab02> .\task2.exe
Enter X (|X| < 1): 0.5
Enter precision eps (e.g. 1e-5): 0.01
Series sum      : 1.9843750000
Exact 1/(1-X)   : 2.0000000000
Absolute error  : 1.562e-02
Relative error  : 7.812e-03
Iterations      : 7
```

```
PS C:\Users\illia\projects\programming-sem1\lab02> ./task2.exe
Enter X (|X| < 1): 0.5
Enter precision eps (e.g. 1e-5): 0.0001
Series sum      : 1.9998779297
Exact 1/(1-X)   : 2.0000000000
Absolute error  : 1.221e-04
Relative error  : 6.104e-05
Iterations      : 14
```

```
PS C:\Users\illia\projects\programming-sem1\lab02> .\task2.exe
Enter X (|X| < 1): 0.5
Enter precision eps (e.g. 1e-5): 0.000001
Series sum      : 1.9999980927
Exact 1/(1-X)   : 2.0000000000
Absolute error  : 1.907e-06
Relative error  : 9.537e-07
Iterations      : 20
```

Зведена таблиця збіжності:

X	ε	Сума ряду	Точне значення	Абс. похибка	Відн. похибка	Ітерацій
0.5	10^{-2}	1.9843750000	2.0000000000	$1.562 \cdot 10^{-2}$	$7.812 \cdot 10^{-3}$	7
0.5	10^{-4}	1.9998779297	2.0000000000	$1.221 \cdot 10^{-4}$	$6.104 \cdot 10^{-5}$	14
0.5	10^{-6}	1.9999980927	2.0000000000	$1.907 \cdot 10^{-6}$	$9.537 \cdot 10^{-7}$	20

6.4. Завдання 2. Вихід за область збіжності

```
PS C:\Users\illia\projects\programming-sem1\lab02> .\task2.exe
Enter X (|X| < 1): 1.5
Error: X is outside the convergence region |X| < 1.
```

7. Висновки

1. Ряд $1/(1 - X) = \Sigma X^n$ збігається лише при $|X| < 1$, і швидкість збіжності залежить від значення X. При X = 0.5 для досягнення точності $\varepsilon = 10^{-6}$ потрібно 20 ітерацій. Чим ближче |X| до межі області збіжності, тим повільніше спадають доданки і тим більше ітерацій потрібно для досягнення тієї самої точності.

2. Кожне зменшення ε у 100 разів додає приблизно 6-7 ітерацій (для X = 0.5: 7, 14, 20), тобто кількість ітерацій росте приблизно пропорційно логарифму 1/ε. Отримані похибки ($1.562 \cdot 10^{-2}$, $1.221 \cdot 10^{-4}$, $1.907 \cdot 10^{-6}$) не перевищують заданої точності.

3. Рекурентне обчислення доданків через співвідношення $a_n = a_{n-1} \cdot X$ потребує одного множення на ітерацію. Пряме обчислення X^n функцією pow() на кожному кроці було б повільнішим, а для рядів із факторіалами ще й призводило б до переповнення. Тому рекурентний підхід ефективніший.

4. Програми містять захист від некоректних даних (перевірка значення scanf, Xstart < Xend, dX > 0, |X| < 1, ε > 0) і лічильник ітерацій для запобігання зациклення, тому не завершуються аварійно.

5. Мету роботи досягнуто: опановано оператори розгалуження, цикли, чисельні методи обчислення функцій рядами та оформлення технічної документації.